

Mobility Lens — Report

CS501 E1 — Mobile Application Development

Package: `com.ankitapatra.mobilitylens`

Test Device: Pixel 7 Emulator, API 35

SDK Configuration: `minSdk 24`, `targetSdk 37`, `compileSdk 37`

Name: Ankita Patra

BU ID: U38177365

1. Purpose and Overview

Mobility Lens is a single-screen Android application developed using Kotlin, Jetpack Compose, and Material 3. Its purpose is to demonstrate how mobile application development differs from traditional desktop development through six mobility dimensions: Input and Interaction; Screen Size, Orientation, and Density; Lifecycle and Resource Constraints; Context Awareness; Usage Patterns; and Security and Privacy Expectations.

Users navigate through the six dimensions using Previous and Next buttons. Each dimension presents a mobile constraint and its corresponding developer implication. The application also includes a “Try It Yourself” section where users enter an application or feature name and analyze it in relation to the currently selected mobility dimension. Empty input produces visible validation feedback, while valid input generates a dynamic analysis result.

2. SDK Settings

The project uses **`minSdk 24`**, **`targetSdk 37`**, and **`compileSdk 37`**. The `minSdk` defines the oldest Android API level on which Mobility Lens can run. The `targetSdk` identifies the Android API level the application targets and determines how the system applies platform behavior and compatibility rules. The `compileSdk` specifies the Android API level used to compile the project and determines which Android APIs are available during development. The application was tested using a **Pixel 7 emulator running API 35**.

3. Key Project Files

MainActivity.kt, located in `app/src/main/java/com/ankitapatra/mobilitylens/`, is the application entry point and initializes the Jetpack Compose interface. AndroidManifest.xml, located in `app/src/main/`, declares the application and MainActivity as its launcher activity.

The app-level `app/build.gradle.kts` defines Android configuration, including the application ID and SDK settings, and declares module dependencies. `gradle/libs.versions.toml` acts as the version catalog and centralizes dependency and plugin versions.

`app/src/main/res/values/strings.xml` stores user-facing labels and messages as Android string resources instead of unnecessarily hardcoding them in the Compose interface.

The application code is further separated into `data`, `ui`, `ui/components`, and `ui/theme` packages to keep the data model, screen logic, reusable UI components, and visual theme organized.

4. Rotation Observation

I tested Mobility Lens in both portrait and landscape orientations. The vertically scrollable Compose layout kept the content accessible when the available vertical space decreased in landscape mode. The selected dimension and entered text use `remember`. When the emulator was rotated, the Activity was recreated, causing the selected dimension to return to Dimension 1 and clearing the entered text. This demonstrated that state retained during recomposition does not automatically survive Activity recreation.

5. Mobile Design Decisions

I used a single-column, vertically scrollable Compose layout because mobile devices have limited screen width and may be used in both portrait and landscape orientations. Unlike a desktop interface that can distribute content across a large screen, the single-column layout keeps information readable and touch controls accessible on a phone. Vertical scrolling also ensures that content remains accessible when the available screen height decreases in landscape orientation.

Screenshots

Figure 1 — Running application on the first mobility dimension

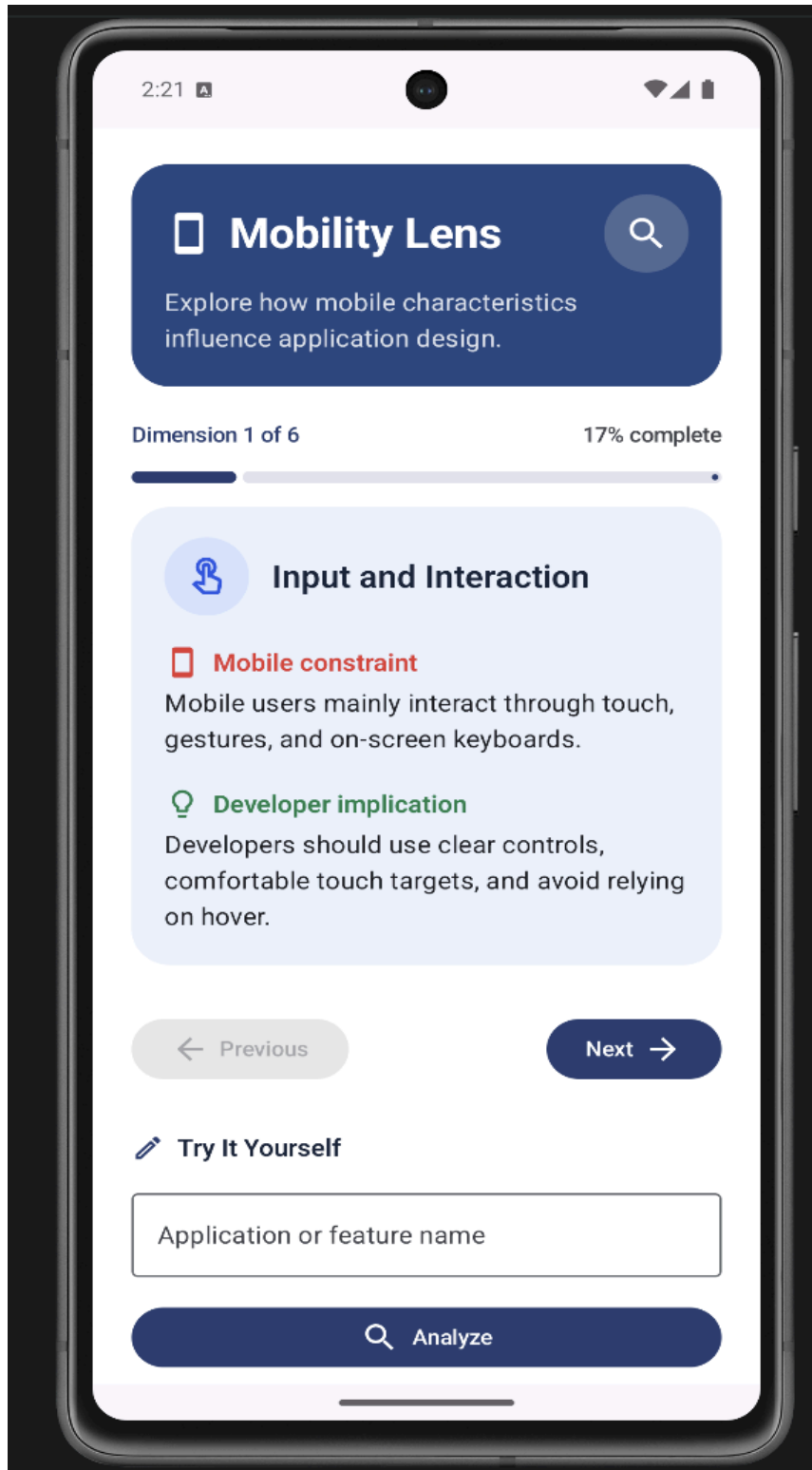


Figure 2 — Validation response to blank input

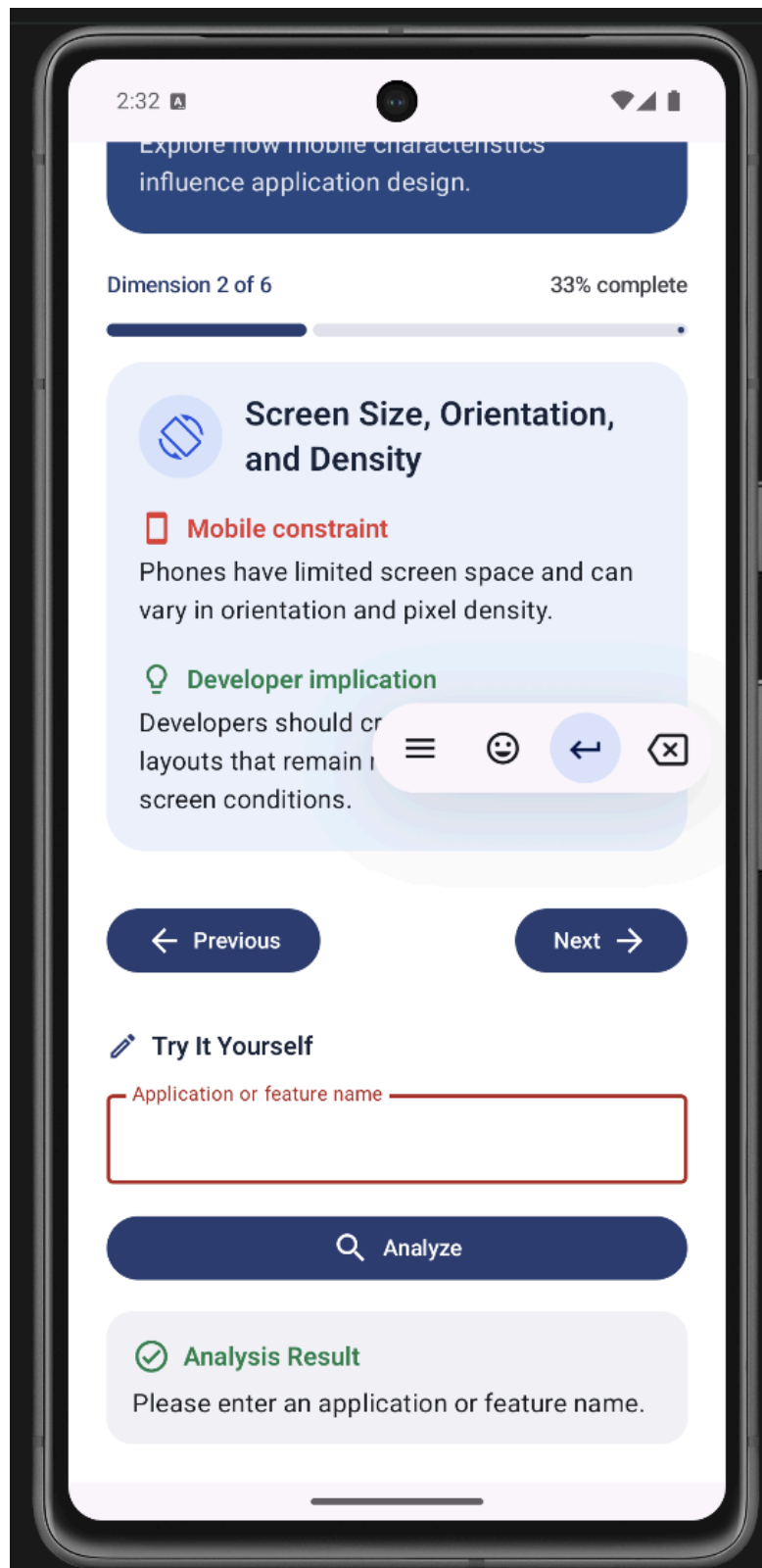


Figure 3 — Successful response to valid input

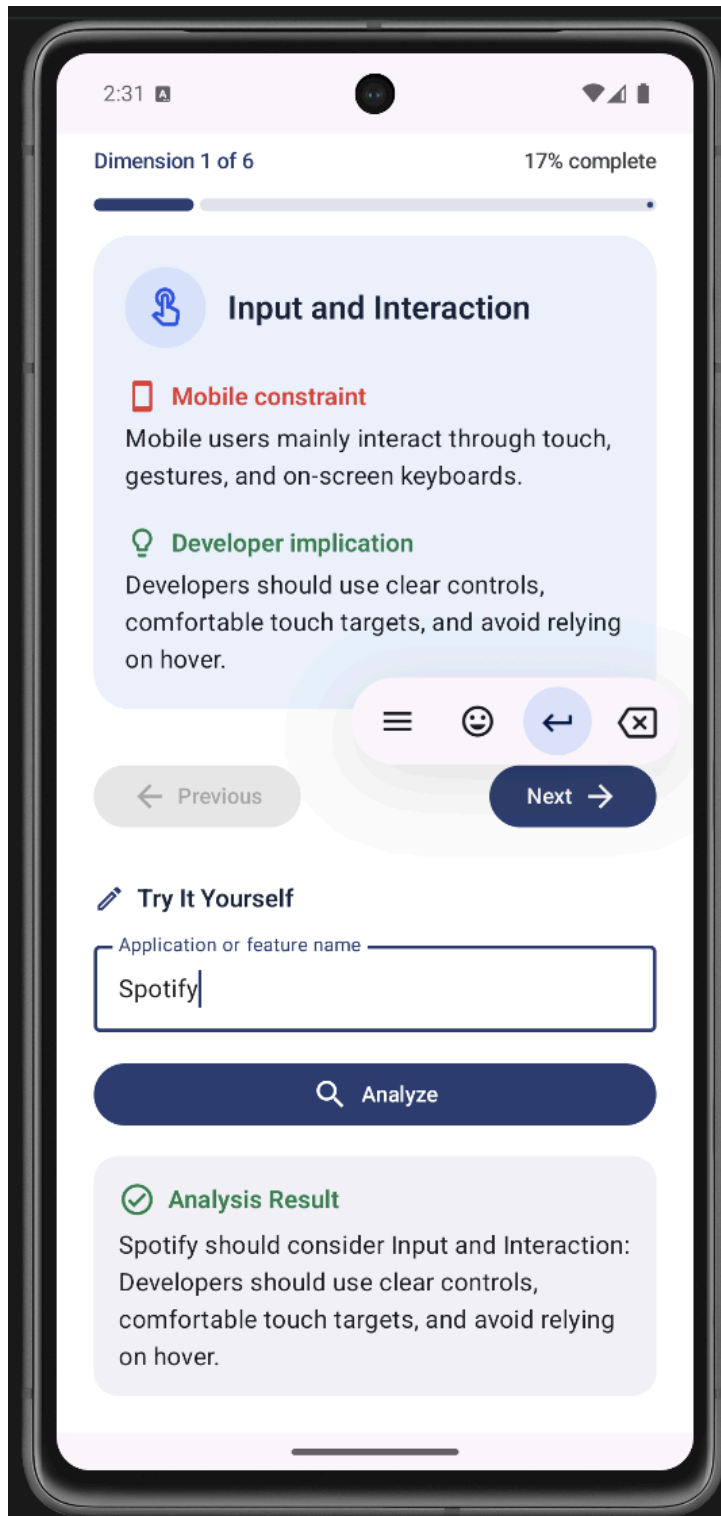
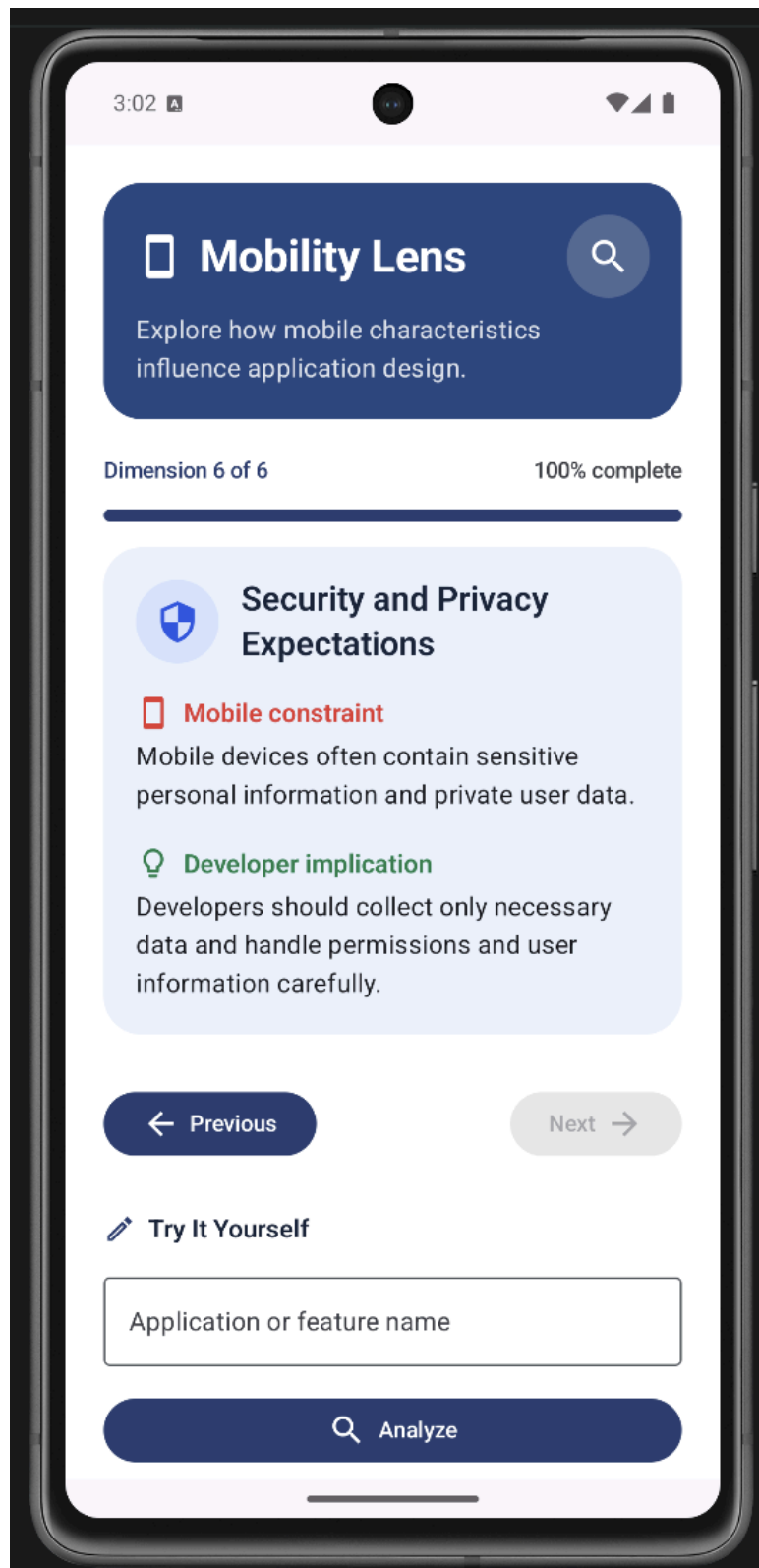


Figure 4 — Different mobility dimension after user interaction



6. AI Assistance Disclosure

Generative AI was used as a limited development aid for guidance on Android project setup, Jetpack Compose concepts, UI design suggestions and documentation organization. AI-provided suggestions and code examples were reviewed and adapted before use. The application was manually implemented, executed, tested, and verified in Android Studio.